

一、单项选择题

1.C 2.A 3.C 4.D 5.B 6.C 7.C 8.B 9.C 10.B
11.A 12.A 13.C 14.B 15.D

二、判断题

1.× 2.× 3.× 4.× 5.√ 6.√ 7.√ 8.× 9.× 10.√ 11.√
12.√ 13.√ 14.√ 15.√

三、填空题

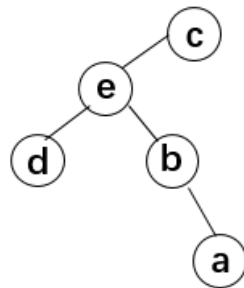
- 1 . 对数据的运算
- 2 . 时间复杂度、 空间复杂度
- 3 . 链式
- 4 . 3
- 5 . 求余
- 6 . $(Q.rear + 1) \% maxSize = Q.front$
- 7 . $\lfloor \log_2 n \rfloor$ 或 $\lfloor \log_2(n + 1) \rfloor$
- 8 . 2^{n-1}
- 9 . 先序、中序
- 10 . 递归
- 11 . 稳定
- 12 . 二路归并
- 13 . 二叉排序树

四、简答题

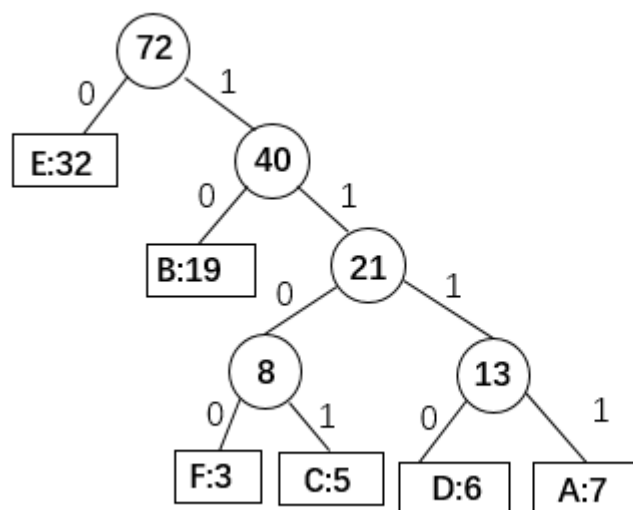
1. 可以构成四种。它们可以同属于一个集合，构成集合结构；它们之间可以只存在一对一的关系构成线性结构；它们之间可以有一对多的关系，构成树形结构；这些元素之间也可以存在多对多的关系构成图形结构。

2.

后序: d a b e c



3.

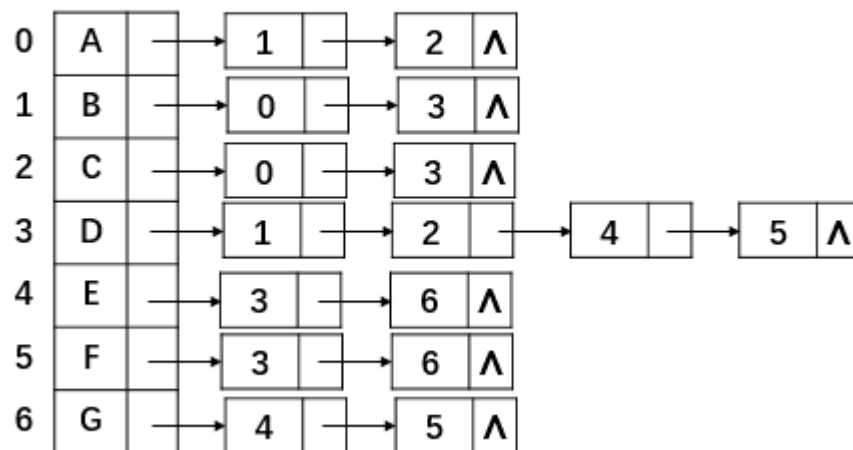


哈夫曼编码

E	0
B	10
F	1100
C	1101
D	1110
A	1111

4.

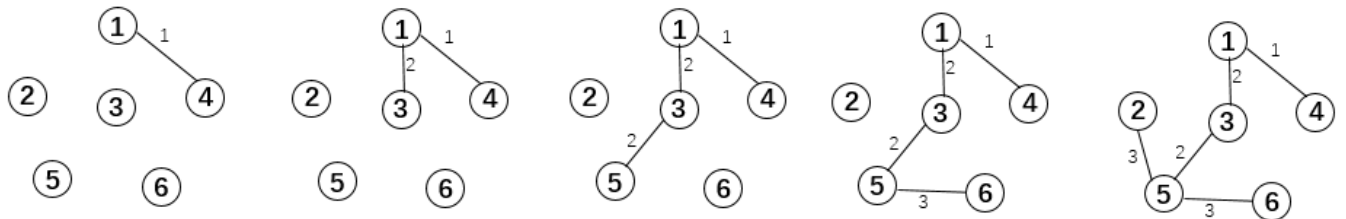
邻接表



深度: A B D C E G F

广度: A B C D E F G

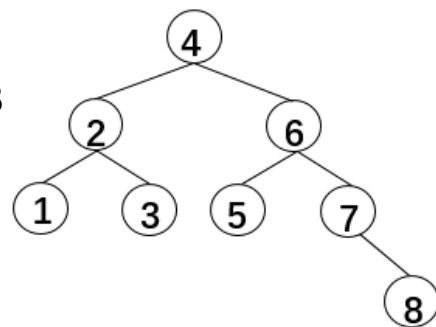
5.



6.假定待查找的 8 个元素为 1、2、3、4、5、6、7、8，则折半查找判定树为

$$ASL_{\text{成功}} = (1+2 \times 2+3 \times 4+4)/8 = 21/8$$

$$ASL_{\text{失败}} = (3 \times 7+4 \times 2)/9 = 29/9$$



五、算法设计题

```
1. 1 typedef struct LNode //定义单链表结点类型
    2 {
    3     int data; //数据域
    4     struct LNode *next; //指针域
    5 } LNode, *LinkList;
    6 void Insert(LinkList &L, int x)
    7 {
    8     LNode *p, *s;
    9     p = L;
   10     s = (LNode*)malloc(sizeof(LNode)); //申请新结点, 其数据域为 x
   11     s->data = x;
   12     s->next = NULL;
   13     while(p->next != NULL)
   14     {
   15         if(p->next->data >= x) //找到插入位置
   16         {
   17             s->next = p->next; //将新结点插入到表中
   18             p->next = s;
   19         }
   20         else
   21             //如果链表为空或链表中所有结点值都大于 x, 则将新结点插在链表最后
   22             p = p->next;
   23     }
   24     p->next = s;
   25 }
```

2.

```
1 typedef struct BTreeNode
2 {
3     datatype data;
4     struct BTreeNode *lchild, *rchild;
5 }BTreeNode, *BTree
261)
7 int count(BTree t)
8 {
9     int n1, n2;
10    if(T==NULL)
11        return 0;
12    else
13    {
14        n1 = count(T->lchild); //左子树中结点数, 返回给n1
15        n2 = count(T->rchild);
16
17        return n1+n2+1;
18    }
19 }
```

3.

```
1 void Insert_Sort(int A[], int n)
2 {
3     int i, j;
4     int temp;
5     for(i=1; i<n; ++i)
6     {
7         temp = A[i]; //将待插关键字暂存于temp中
8         j = i-1;
9
10        /* 下面这个循环完成了从待排关键字之前的关键字开始扫描,
11        如果小于待排关键字, 则后移一位 */
12        while(j>=0 && temp>A[j])
13        {
14            A[j+1] = A[j];
15            --j;
16        }
17        A[j+1] = temp;
18    }
19 }
```